

COURSE INFORMATION

Description:

Learners are introduced to algorithms as structured approaches to solving problems. The course focuses on analyzing problems, selecting appropriate strategies, and implementing solutions using common data structures. Students explore how different approaches affect performance through basic analysis techniques and comparison of solutions. Emphasis is placed on reasoning, debugging, and refining solutions both manually and with AI-assisted tools. By the end of the course, learners are able to evaluate, adapt, and explain algorithmic approaches used to solve problems effectively

Career Cluster: Information Technology

Instructional Level: A.A.S. - Associate in Applied Science

Total Credits: 3

Total Hours: 72

PRE/COREQUISITES

- Prerequisite: 10-152-117 Python Programming

TEXTBOOKS

50 Algorithms Every Programmer Should Know: Tackle computer science challenges with classic to modern algorithms in machine learning, software design, data systems, and cryptography - 2nd ed. Edition - Imran Ahmadad

COURSE COMPETENCIES

1. Analyze problems to define algorithmic solutions

Domain: Cognitive

Level: Analyzing Status: WIP

Assessment Strategies

- 1.1. Project assignments, lab activities, and quiz

Criteria

Performance will meet expectations when:

- 1.1. Decompose problems into clear steps
- 1.2. Identify inputs, outputs, constraints, assumptions, and edge cases
- 1.3. Distinguish between clear and ambiguous problem statements
- 1.4. Represent solution logic using pseudocode, diagrams, traces, or code
- 1.5. Explain why a proposed approach fits the problem

Learning Objectives

- 1.a. Define an algorithm and its key characteristics

- 1.b. Explain problem decomposition and abstraction
- 1.c. Identify problem constraints and assumptions
- 1.d. Describe algorithmic solutions before implementation

2. Select and use data structures to support algorithmic solutions

Domain: Cognitive Level: Applying Status: WIP

Assessment Strategies

2.1. Project assignments, lab activities, and quiz

Criteria

Performance will meet expectations when:

- 2.1. Use lists, dictionaries, sets, stacks, queues, trees, and graphs at an introductory level
- 2.2. Match data structures to access, lookup, ordering, traversal, or membership needs
- 2.3. Explain how a data structure affects solution clarity and performance
- 2.4. Represent simple relationships or workflows using appropriate structures
- 2.5. Select structures based on problem requirements and constraints

Learning Objectives

- 2.a. Identify common data structures used in algorithmic solutions
- 2.b. Explain basic tradeoffs among common data structures
- 2.c. Describe how representation affects algorithm design
- 2.d. Use Python structures to support algorithm implementation

3. Implement and test introductory algorithms using Python

Domain: Cognitive Level: Applying Status: WIP

Assessment Strategies

3.1. Project assignments, lab activities, and quiz

Criteria

Performance will meet expectations when:

- 3.1. Implement introductory searching, sorting, recursive or iterative, and graph traversal algorithms
- 3.2. Translate algorithmic logic into readable Python code
- 3.3. Test solutions using normal cases, edge cases, and expected outcomes
- 3.4. Identify and correct logical errors
- 3.5. Use libraries appropriately when implementation is not the learning target

Learning Objectives

- 3.a. Describe how core programming constructs support algorithms
- 3.b. Implement algorithmic steps in Python
- 3.c. Explain the relationship between pseudocode and executable code
- 3.d. Apply testing to verify algorithm correctness

4. Evaluate correctness, efficiency, and tradeoffs

Domain: Cognitive Level: Evaluating Status: WIP

Assessment Strategies

4.1. Project assignments, lab activities, and quiz

Criteria

Performance will meet expectations when:

- 4.1. Verify whether an algorithm produces expected results
- 4.2. Compare multiple solutions to the same problem
- 4.3. Apply introductory Big-O reasoning to describe growth behavior
- 4.4. Discuss tradeoffs among correctness, simplicity, readability, efficiency, and maintainability
- 4.5. Use timing data, traces, diagrams, or other evidence to support evaluation

Learning Objectives

- 4.a. Define correctness in an algorithmic context
- 4.b. Explain common growth patterns such as constant, linear, quadratic, and logarithmic behavior
- 4.c. Identify common sources of inefficiency or incorrect assumptions
- 4.d. Describe tradeoffs in algorithm selection

5. Connect algorithmic thinking to AI, analytics, data modeling, and responsible tool use

Domain: Cognitive

Level: Evaluating

Status: WIP

Assessment Strategies

- 5.1. Project assignments, lab activities, quiz, and reflection or explanation artifacts

Criteria

Performance will meet expectations when:

- 5.1. Recognize how similarity, clustering, recommendation, hashing, and representation support AI and data systems
- 5.2. Explain how assumptions, bias, incomplete data, or poor representation can affect algorithmic outcomes
- 5.3. Compare manual, library-supported, and AI-assisted solutions when appropriate
- 5.4. Identify limitations, inaccuracies, or hidden assumptions in AI-generated algorithmic output
- 5.5. Communicate algorithmic choices clearly to technical and non-technical audiences

Learning Objectives

- 5.a. Describe selected algorithmic foundations of AI and data systems
- 5.b. Explain why algorithms require validation and interpretation
- 5.c. Identify responsible uses and limits of AI-assisted development
- 5.d. Communicate solution choices and tradeoffs clearly